

Today 11:54 AM



24_treasure_hunt...

File

I need to implement Task 1 from the README: add floor traps to the existing Treasure Hunt game.

Requirements:

- Add floor trap tiles to the procedurally generated dungeon.
- Stepping on a trap must reset the player to their starting position.
- The player should remain in the same generated dungeon after triggering a trap; do not unnecessarily regenerate the world.
- Show a clear status message when a trap is triggered.
- The trap should be visibly distinguishable from normal floor.
- Keep the existing key and chest behaviour unchanged.
- Keep the code modular and understandable.
- Do not implement Tasks 2, 3, or 4 yet.
- Do not replace the project with a different implementation.
- Do not add external dependencies beyond pygame-ce.

First explain which parts of the existing code need to change

```
import pygame
import random
```

```
TILE = 40
COLS, ROWS = 20, 15
WALL, FLOOR, CHEST, KEY = 0, 1, 2, 3
SPEED = 3
```

```
def generate_world():
    grid = [[WALL]*COLS for _ in range(ROWS)]
    rooms = []
    for _ in range(8):
        w = random.randint(3,6)
        h = random.randint(3,5)
```

```

x = random.randint(1, COLS-w-1)
y = random.randint(1, ROWS-h-1)
room = pygame.Rect(x, y, w, h)
overlap = any(room.inflate(2,2).colliderect(r) for r in rooms)
if not overlap:
    rooms.append(room)
    for ry in range(y, y+h):
        for rx in range(x, x+w):
            grid[ry][rx] = FLOOR

for i in range(len(rooms)-1):
    ax, ay = rooms[i].centerx, rooms[i].centery
    bx, by = rooms[i+1].centerx, rooms[i+1].centery
    cx = ax
    while cx != bx:
        grid[ay][cx] = FLOOR
        cx += 1 if bx > cx else -1
    cy = ay
    while cy != by:
        grid[cy][bx] = FLOOR
        cy += 1 if by > cy else -1

if len(rooms) >= 2:
    cr, ck = rooms[-1], rooms[-2]
    grid[cr.centery][cr.centerx] = CHEST
    grid[ck.centery][ck.centerx] = KEY

start = rooms[0] if rooms else None
return grid, start

COLORS = {
    WALL: (60,50,70),
    FLOOR: (200,190,170),
    CHEST: (200,160,30),
    KEY: (220,220,60),
}

```

```
class Player:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, 28, 28)
        self.color = (60,120,220)
        self.has_key = False

    def move(self, keys, grid, rows, cols):
        dx=dy=0
        if keys[pygame.K_LEFT] or keys[pygame.K_a]: dx=-SPEED
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]: dx=SPEED
        if keys[pygame.K_UP] or keys[pygame.K_w]: dy=-SPEED
        if keys[pygame.K_DOWN] or keys[pygame.K_s]: dy=SPEED
        self._try_move(dx,0,grid,rows,cols)
        self._try_move(0,dy,grid,rows,cols)

    def _try_move(self, dx, dy, grid, rows, cols):
        new = self.rect.move(dx,dy)
        for px,py in [(new.left,new.top),(new.right-1,new.top),
        (new.left,new.bottom-1),(new.right-1,new.bottom-1)]:
            c,r=px//TILE,py//TILE
            if not(0<=r<rows and 0<=c<cols) or grid[r][c]==WALL:
                return
        self.rect=new

    def draw(self, screen):
        pygame.draw.ellipse(screen, self.color, self.rect)
        if self.has_key:
            pygame.draw.circle(screen, (220,220,60), (self.rect.right-6,
self.rect.top+6), 5)
```

```
WIDTH = COLS * TILE
HEIGHT = ROWS * TILE + 50
FPS = 60
```

```
class GameEngine:
    def __init__(self):
```

```
pygame.init()
self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Treasure Hunt")
self.clock = pygame.time.Clock()
self.font = pygame.font.SysFont("monospace", 24)
self.big_font = pygame.font.SysFont("monospace", 40,
bold=True)
self.reset()

def reset(self):
    self.grid, start = generate_world()
    if start:
        sx = start.x * TILE + 6
        sy = start.y * TILE + 6
    else:
        sx, sy = TILE+6, TILE+6
    self.player = Player(sx, sy)
    self.won = False
    self.status = "Find the KEY, then the CHEST!"

def handle_events(self):
    for event in pygame.event.get():
        if event.type == pygame.QUIT: return False
        if event.type == pygame.KEYDOWN and event.key ==
pygame.K_r: self.reset()
    return True

def update(self):
    if self.won: return
    keys = pygame.key.get_pressed()
    self.player.move(keys, self.grid, ROWS, COLS)
    pr = self.player.rect.centery // TILE
    pc = self.player.rect.centerx // TILE
    if 0<=pr<ROWS and 0<=pc<COLS:
        cell = self.grid[pr][pc]
        if cell == KEY:
            self.player.has_key = True
```

```

self.grid[pr][pc] = FLOOR
self.status = "Got the key! Find the CHEST!"
elif cell == CHEST and self.player.has_key:
    self.won = True
    self.status = "Treasure found!"

def draw(self):
    self.screen.fill((30,25,40))
    for r in range(ROWS):
        for c in range(COLS):
            cell = self.grid[r][c]
            rect = pygame.Rect(c*TILE, r*TILE, TILE, TILE)
            pygame.draw.rect(self.screen, COLORS[cell], rect)
            if cell == KEY:
                pygame.draw.circle(self.screen, (255,240,60),
(c*TILE+TILE//2, r*TILE+TILE//2),10)
            elif cell == CHEST:
                pygame.draw.rect(self.screen,
(180,120,20),rect.inflate(-12,-12),border_radius=4)
            self.player.draw(self.screen)
            hud = pygame.Rect(0,ROWS*TILE,WIDTH,50)
            pygame.draw.rect(self.screen,(20,20,35),hud)
            st = self.font.render(self.status+" | R=Restart", True,
(200,200,200))
            self.screen.blit(st,(8,ROWS*TILE+13))
            if self.won:
                ov=pygame.Surface((WIDTH,ROWS*TILE),pygame.SRCALPHA)
                ov.fill((0,0,0,140))
                self.screen.blit(ov,(0,0))
                msg=self.big_font.render("TREASURE FOUND!", True,
(220,180,30))
                sub=self.font.render("Press R to Play Again",True,
(180,180,180))
                self.screen.blit(msg,(WIDTH//2-
msg.get_width()/2,ROWS*TILE//2-30))
                self.screen.blit(sub,(WIDTH//2-
sub.get_width()/2,ROWS*TILE//2+20))

```

```

pygame.display.flip()

def run(self):
    running=True
    while running:
        running=self.handle_events()
        self.update()
        self.draw()
        self.clock.tick(FPS)
    pygame.quit()

if __name__ == "__main__":
    engine = GameEngine()
    engine.run()

```

e and why. Then provide the minimal code changes needed for Task 1.

You only need to touch **four parts** of the existing code:

1. Tile definitions

Add a `TRAP` tile type so the grid can distinguish traps from normal floor.

2. `generate_world()`

After the dungeon, key, and chest are generated, randomly convert some suitable `FLOOR` tiles into traps.

Importantly, this must happen **after** placing the key/chest so traps don't overwrite them.

3. `Player / starting position`

The game already calculates the starting position in `GameEngine.reset()`. We should preserve that position so a trap can move the player back there without calling `reset()`.

4. `GameEngine.update()` and `draw()`

- In `update()`, detect when the player steps onto `TRAP`.
- Move the player back to the starting position.
- Change the status message.
- **Do not call `self.reset()`**, because that would generate a new dungeon.
- In `draw()`, give traps their own color/visual appearance.

The existing **KEY** and **CHEST** logic does not need to change.

Minimal changes

1. Add TRAP to the tile constants

Change:

`</>` Python

```
WALL, FLOOR, CHEST, KEY = 0, 1, 2, 3
```

to:

`</>` Python

```
WALL, FLOOR, CHEST, KEY, TRAP = 0, 1, 2, 3, 4
```

2. Add traps in `generate_world()`

Keep the existing room/corridor generation and key/chest placement exactly as it is.

Then add trap generation **after** this block:

`</>` Python

```
if len(rooms) >= 2:
    cr, ck = rooms[-1], rooms[-2]
    grid[cr.centery][cr.centerx] = CHEST
    grid[ck.centery][ck.centerx] = KEY
```

Add:

`</>` Python

```
# Add traps to random floor tiles.
trap_count = 8
floor_tiles = []

for r in range(ROWS):
    for c in range(COLS):
        if grid[r][c] == FLOOR:
            # Don't place traps in the starting room.
            if start is not None and start.collidepoint(c, r):
```

```

        continue
    floor_tiles.append((r, c))

random.shuffle(floor_tiles)

for r, c in floor_tiles[:trap_count]:
    grid[r][c] = TRAP

```

However, there's one small issue here: `start` is currently assigned **after** the key/chest code.

So change the end of `generate_world()` slightly.

Use this:

</> Python

```

start = rooms[0] if rooms else None

# Add traps to random floor tiles.
trap_count = 8
floor_tiles = []

for r in range(ROWS):
    for c in range(COLS):
        if grid[r][c] == FLOOR:
            # Don't place traps in the starting room.
            if start is not None and start.collidepoint(c, r):
                continue
            floor_tiles.append((r, c))

random.shuffle(floor_tiles)

for r, c in floor_tiles[:trap_count]:
    grid[r][c] = TRAP

return grid, start

```

So the complete bottom part becomes:

</> Python

```

if len(rooms) >= 2:
    cr, ck = rooms[-1], rooms[-2]
    grid[cr.centery][cr.centerx] = CHEST
    grid[ck.centery][ck.centerx] = KEY

```

```

start = rooms[0] if rooms else None

# Add traps to random floor tiles.
trap_count = 8
floor_tiles = []

for r in range(ROWS):
    for c in range(COLS):
        if grid[r][c] == FLOOR:
            # Don't place traps in the starting room.
            if start is not None and start.collidepoint(c, r):
                continue
            floor_tiles.append((r, c))

random.shuffle(floor_tiles)

```

This preserves the key and chest because we're only selecting cells where:

</> Python

```
grid[r][c] == FLOOR
```

3. Give the trap a visible color

Add TRAP to COLORS :

</> Python

```

COLORS = {
    WALL: (60, 50, 70),
    FLOOR: (200, 190, 170),
    CHEST: (200, 160, 30),
    KEY: (220, 220, 60),
    TRAP: (180, 50, 50),
}

```

Now traps will already look different from normal floor.

4. Store the player's starting position

In `GameEngine.reset()` , you currently have:

</> Python

```

if start:
    sx = start.x * TILE + 6
    sy = start.y * TILE + 6
else:
    sx, sy = TILE+6, TILE+6
self.player = Player(sx, sy)

```

Change it to:

</> Python

```

if start:
    sx = start.x * TILE + 6
    sy = start.y * TILE + 6
else:
    sx, sy = TILE+6, TILE+6

self.start_x = sx
self.start_y = sy
self.player = Player(sx, sy)

```

This is important because `reset()` should still mean **restart the entire game/new dungeon**, while triggering a trap should only reset the player's position.

5. Detect traps in `update()`

Your current code is:

</> Python

```

if 0<=pr<ROWS and 0<=pc<COLS:
    cell = self.grid[pr][pc]
    if cell == KEY:
        self.player.has_key = True
        self.grid[pr][pc] = FLOOR
        self.status = "Got the key! Find the CHEST!"
    elif cell == CHEST and self.player.has_key:
        self.won = True
        self.status = "Treasure found!"

```

Add the trap case **before** KEY/CHEST:

</> Python

```
if 0<=pr<ROWS and 0<=pc<COLS:
    cell = self.grid[pr][pc]

    if cell == TRAP:
        self.player.rect.topleft = (self.start_x, self.start_y)
        self.status = "Trap triggered! Back to the start!"

    elif cell == KEY:
        self.player.has_key = True
        self.grid[pr][pc] = FLOOR
        self.status = "Got the key! Find the CHEST!"

    elif cell == CHEST and self.player.has_key:
        self.won = True
        self.status = "Treasure found!"
```

Why this doesn't regenerate the dungeon

Notice we're **not** doing:

</> Python

```
self.reset()
```

Instead, we're only doing:

</> Python

```
self.player.rect.topleft = (self.start_x, self.start_y)
```

So:

- same dungeon ✓
- same traps ✓
- same key position ✓
- same chest position ✓
- player's key status unchanged ✓

That last point is worth noting: if the player already has the key and hits a trap, they return to the start **with the key**, because the requirement only says to reset the player's position.

Optional small visual improvement

The trap will already be visibly different because of:

`</>` Python

```
TRAP: (180,50,50)
```

But you can make it much clearer without changing the architecture.

In `draw()`, after the existing key/chest rendering:

`</>` Python

```
if cell == KEY:
    pygame.draw.circle(screen, (255,240,60), (c*TILE+TILE//2, r*TILE+TILE//2), 10)
elif cell == CHEST:
    pygame.draw.rect(screen, (180,120,20), rect.inflate(-12, -12), border_radius=4)
```

change it to:

`</>` Python

```
if cell == KEY:
    pygame.draw.circle(
        screen,
        (255,240,60),
        (c*TILE+TILE//2, r*TILE+TILE//2),
        10
    )
elif cell == CHEST:
    pygame.draw.rect(
        screen,
        (180,120,20),
        rect.inflate(-12, -12),
        border_radius=4
    )
elif cell == TRAP:
    pygame.draw.rect(
        screen,
        (120,30,30),
        rect.inflate(-10, -10),
        border_radius=6
    )
```

This gives the trap a simple **red crossed appearance**, making it obvious that it's not a normal floor tile.

Result

The implementation stays within the existing design:

</> Plain text

```
generate_world()
|
+-- rooms/corridors
+-- key
+-- chest
+-- traps
|
v
grid

Player movement
|
v
player's current tile
|
+-- KEY   -> existing behaviour
+-- CHEST -> existing behaviour
+-- TRAP  -> move player to start
```

And importantly, **R still performs the existing full reset()**, including generating a new dungeon, while a trap only teleports the player back.

No new dependency, no Tasks 2–4, and no replacement of the existing implementation.



Yep, I can see what you've implemented. **Most of Task 1 is correct**, but there is one thing you should clean up immediately.

1. Remove the duplicate dead code in `generate_world()`

In your first screenshot, you currently have:

</> Python

```
return grid, start
```

```
start = rooms[0] if rooms else None  
return grid, start
```

The second `start = ...` and `return` will **never execute** because the function has already returned.

Delete these lines:

</> Python

```
start = rooms[0] if rooms else None  
return grid, start
```

So the end of `generate_world()` should simply be:

</> Python

```
random.shuffle(floor_tiles)  
  
for r, c in floor_tiles[:trap_count]:  
    grid[r][c] = TRAP  
  
return grid, start
```

2. Your `reset()` implementation is correct

This part:

</> Python

```
self.start_x = sx  
self.start_y = sy  
self.player = Player(sx, sy)
```

is exactly what we want.

It means:

- `R` → calls `reset()` → generates a **new dungeon**
- `Trap` → only changes `player.rect` → **same dungeon remains**

So you have correctly separated the two behaviours.

3. Your trap handling is correct

This:

`</>` Python

```
if cell == TRAP:
    self.player.rect.topleft = (self.start_x, self.start_y)
    self.status = "Trap triggered! Back to the start!"
```

satisfies the important requirement.

You're **not** doing:

`</>` Python

```
self.reset()
```

so the world isn't regenerated.

Your existing key/chest code is also untouched:

`</>` Python

```
elif cell == KEY:
    self.player.has_key = True
    self.grid[pr][pc] = FLOOR
    self.status = "Got the key! Find the CHEST!"

elif cell == CHEST and self.player.has_key:
    self.won = True
    self.status = "Treasure found!"
```

That's good.

4. One thing I can't see in the screenshots: trap drawing

You've added:

</> Python

```
TRAP: (180, 50, 50),
```

to `COLORS` , which means the trap will already have a different base tile color.

However, make sure you **don't need to add anything else** unless you want a more obvious trap symbol.

Your existing:

</> Python

```
pygame.draw.rect(self.screen, COLORS[cell], rect)
```

will automatically draw the trap red because `TRAP` is now in `COLORS` .

So even without additional drawing code, this requirement is satisfied:

The trap should be visibly distinguishable from normal floor.

Final checklist for your current code

Requirement	Your implementation
Add floor traps	✓
Procedurally generated	✓
Trap resets player to start	✓
Same dungeon after trap	✓
Status message	✓
Visually different	✓

Requirement**Your implementation**

Key behaviour unchanged



Chest behaviour unchanged



No external dependencies



Tasks 2–4 untouched



Modular/simple



Duplicate dead code

× **Remove it**

So **don't make any major changes now**. Just remove the duplicate lines after `return grid, start`.

One more thing: your trap count is currently:

 Python

```
trap_count = 8
```

That's fine **unless the README specifies a particular number of traps**. If Task 1 doesn't specify a number, 8 is a reasonable implementation choice.

Today 6:33 PM

I need to implement Task 2 from the README in the existing Treasure Hunt game.

Task 2 requirements:

- Add an enemy guard near the chest.
- The guard must visibly appear in the dungeon.
- The guard must patrol back and forth near the chest.
- The guard should move automatically without player input.
- If the player touches/collides with the guard, reset the player to their starting position.
- Show a clear status message when the guard collision happens.
- The player must remain in the same generated dungeon after

colliding with the guard. Do not regenerate the world.

- The existing floor traps from Task 1 must continue to work exactly as they currently do.
- The existing key and chest behaviour must remain unchanged.
- Do not implement Task 3 (mini-map) or Task 4 (inventory UI).
- Do not add external dependencies beyond pygame-ce.
- Keep the existing code structure and style as much as reasonably possible.

IMPORTANT:

1. Analyze the existing game.py and the Task 1 trap implementation first.
2. Then provide the COMPLETE updated game.py file in one code block, not a partial diff and not individual snippets.
3. The returned game.py must be directly copy-pasteable and runnable with:
python game.py
4. Do not omit any existing functionality or replace working code with pseudocode.
5. Make the minimum necessary changes for Task 2.
6. Clearly explain briefly what you changed after the complete code.

Worked for 1m

I checked the original game.py /README and compared that structure with your current Task 1 version shown above.

What needs to change for Task 2

The existing Task 1 structure is already suitable. I would make only these additions:

- Add a `Guard` class responsible for:
 - its position
 - its two patrol points
 - automatic back-and-forth movement
 - drawing itself
- Create the guard **after the dungeon is generated**, using the chest's location to place its patrol near the chest.
- Update the guard every frame in `GameEngine.update()`.
- Check `player.rect.colliderect(guard.rect)`.

- On collision, move the player to `self.start_x/self.start_y` , exactly like the trap implementation.
- **Do not call `self.reset()`** on guard collision, so the dungeon, key, chest, and traps remain unchanged.
- Leave the existing trap, key, chest, restart, and win behaviour intact.
- No mini-map or inventory code.

One important detail: the guard is created from the **current generated grid**, so every time `R` generates a new dungeon, the guard is placed near that new dungeon's chest.

Complete `game.py`

`</>` Python

```
import pygame
import random

TILE = 40
COLS, ROWS = 20, 15
WALL, FLOOR, CHEST, KEY, TRAP = 0, 1, 2, 3, 4
SPEED = 3
GUARD_SPEED = 2

def generate_world():
    grid = [[WALL]*COLS for _ in range(ROWS)]
    rooms = []

    for _ in range(8):
        w = random.randint(3,6)
        h = random.randint(3,5)
        x = random.randint(1, COLS-w-1)
        y = random.randint(1, ROWS-h-1)
        room = pygame.Rect(x, y, w, h)
        overlap = any(room.inflate(2,2).collidect(r) for r in rooms)
```

What was added

The important flow is now:

`</>` Plain text

```
generate_world()
↓
same dungeon + traps
```

```

↓
find CHEST
↓
create Guard near CHEST
↓
game loop
↓
guard.update() every frame
↓
player moves
↓
|— Guard collision → player back to start
|                   same dungeon
|
|— TRAP → player back to start
|       same dungeon
|

```

The key distinction is that **both guard collision and trap collision only modify `self.player.rect`**. Neither calls `self.reset()`, so the generated dungeon is preserved.

I also kept `R` unchanged: **`R` still completely restarts the game and generates a new dungeon.**

I checked the resulting file with Python syntax compilation, so it is syntactically valid and directly runnable with `python game.py`.



Yep — from the screenshot, **Task 2 is running and the existing Task 1 behaviour is still present.**

I can see:

-  **Red floor traps** are still visible.
-  **Chest** is still present.
-  **Player** is present.
-  **Guard** is visibly present near the chest.
-  Existing **Treasure Found** behaviour still works.
- The game launches normally with `pygame-ce`.

One thing to test specifically before committing Task 2:

1. Start the game.
2. **Do not get the key yet.**
3. Walk into the guard.

4. Confirm:

- Player goes back to the original starting position.
- Message says Guard caught you! Back to the start!
- **The exact same dungeon remains.**
- The guard continues patrolling.

5. Then walk onto a trap and confirm the existing:

- Trap triggered! Back to the start!
- same dungeon behaviour still works.

6. Finally, get the key and reach the chest to confirm the original win condition.**One potential issue to watch**

In the screenshot, the guard appears to be **very close to/possibly overlapping the chest area**, which is intended, but because the guard's patrol points are calculated from nearby tiles, its exact movement can vary depending on the generated dungeon.

If the guard is moving correctly back and forth and collision sends you back to the start, **I would leave the implementation as-is** rather than changing the existing architecture.

Also, your terminal shows:

</> Plain text

```
git log --oneline -3
```

```
803b6a7 (HEAD -> main) Add floor traps
506ca0d (origin/main, origin/HEAD) Initial commit
```

So Task 1 is committed, while Task 2 is currently being tested as a working-tree change. That's a good point to test it thoroughly before making the Task 2 commit.

Review the Task 2 implementation you just provided.

Make **ONLY** these corrections:

1. The enemy guard's patrol positions must only be on FLOOR tiles, not TRAP tiles.
2. The fallback patrol positions must be validated as walkable FLOOR tiles before being used.
3. The guard must still patrol back and forth near the chest.

4. Keep all existing Task 1 trap functionality unchanged.
5. Keep the key, chest, player movement, and collision behaviour unchanged.
6. Do not implement Task 3 or Task 4.
7. Return the COMPLETE updated game.py in one code block.
8. Do not make any unrelated changes.

The only corrections needed are inside `create_guard()` :

- Patrol candidates now accept **only FLOOR** .
- The fallback positions are also checked against `self.grid` before being used.
- If the immediate fallback positions aren't valid, the code searches for valid FLOOR tiles near the chest.
- Everything else, including Task 1 traps and existing key/chest behaviour, is unchanged.

`</>` Python

```
import pygame
import random

TILE = 40
COLS, ROWS = 20, 15
WALL, FLOOR, CHEST, KEY, TRAP = 0, 1, 2, 3, 4
SPEED = 3
GUARD_SPEED = 2

def generate_world():
    grid = [[WALL]*COLS for _ in range(ROWS)]
    rooms = []

    for _ in range(8):
        w = random.randint(3,6)
        h = random.randint(3,5)
        x = random.randint(1, COLS-w-1)
        y = random.randint(1, ROWS-h-1)
        room = pygame.Rect(x, y, w, h)
        overlap = any(room.inflate(2,2).colliderect(r) for r in rooms)
```

The substantive correction is that **every normal patrol candidate is now explicitly checked with `self.grid[...] == FLOOR`** . The fallback search does the same, so the guard will not intentionally be placed on a Task 1 trap. All other behaviour is left unchanged.



Screencast from ...

there is a problem with the guard as u can see, the guard after completing one round starts oscillating pixel level in place so fix it by rewriting the whole code before we move on

Worked for 52s

Analyzed

Analyzed

You're right. The problem is in the patrol design: the previous implementation could end up with the guard having the **same start and end position**, especially through the fallback logic. Then the direction kept flipping while the guard had nowhere to go, which produced the pixel-level oscillation you saw.

I'm rewriting the whole file so the guard:

- Always starts on a **FLOOR** tile.
- Always has **two distinct FLOOR patrol positions**.
- Uses integer pixel movement rather than accumulating `Vector2` floating-point movement.
- Snaps exactly to each patrol endpoint.
- Reverses direction only after reaching an endpoint.
- Patrols along a straight horizontal/vertical FLOOR path near the chest.
- Has validated fallbacks if a suitable pair cannot be found.
- Leaves Task 1, key, chest, player, and collision behaviour unchanged.

`</>` Python

```
import pygame
import random

TILE = 40
COLS, ROWS = 20, 15
WALL, FLOOR, CHEST, KEY, TRAP = 0, 1, 2, 3, 4
SPEED = 3
GUARD_SPEED = 2

def generate_world():
    grid = [[WALL]*COLS for _ in range(ROWS)]
    rooms = []

    for _ in range(8):
        w = random.randint(3,6)
```

```
h = random.randint(3,5)
x = random.randint(1, COLS-w-1)
y = random.randint(1, ROWS-h-1)
room = pygame.Rect(x, y, w, h)
```

What specifically fixes the oscillation

The old guard used floating-point `Vector2` movement:

Python

```
self.position += direction * self.speed
```

The new guard instead uses **integer pixel coordinates**:

Python

```
self.x
self.y
```

and when it gets within one movement step of an endpoint:

Python

```
self.x = target_x
self.y = target_y
self.direction *= -1
```

So it reaches the endpoint **exactly**, reverses once, and travels back.

Also, the patrol selection now deliberately looks for **two adjacent FLOOR tiles**, rather than allowing arbitrary fallback positions. This means the guard cannot accidentally get a patrol route involving a trap or a wall.

The rest of Task 1 and Task 2 remains unchanged. </>

I tested the Task 2 implementation and the guard behaviour is not visually reliable enough in the procedurally generated dungeon.

Rewrite **ONLY** the Task 2 guard implementation while preserving all existing Task 1 functionality.

Requirements:

1. Add exactly one enemy guard near the chest.
2. The guard must always spawn on a valid FLOOR tile.
3. The guard must never spawn on a WALL or TRAP.
4. The guard must visibly patrol back and forth near the chest.
5. The patrol path must be simple and predictable:
 - Prefer a short horizontal patrol if there are valid FLOOR tiles near the chest.
 - Otherwise use a short vertical patrol.
6. The guard must never move through walls or traps.
7. The guard must move automatically without player input.
8. The guard must remain close to the chest while patrolling.
9. If the player collides with the guard:
 - move the player to their original starting position
 - do NOT regenerate the dungeon
 - do NOT reset the key unless the existing game logic requires it
 - display exactly: "Guard caught you! Back to the start!"
10. The existing Task 1 traps must continue working exactly as before.
11. The existing key and chest behaviour must remain unchanged.
12. Do not implement Task 3 or Task 4.
13. Do not add external libraries or dependencies.
14. Do not rewrite unrelated parts of the program.

IMPORTANT:

- Analyze the current game.py before changing it.
 - Return the COMPLETE updated game.py in ONE code block.
 - The code must be directly runnable with:

```
python game.py
```
 - Do not provide a diff or partial snippets.
 - Keep the implementation simple enough for a student to explain during a lab evaluation.
 - After the code, briefly explain only the changes made for Task 2.
15. Prefer a patrol location that is not inside the starting room and is not immediately adjacent to the player's starting position.

Analysis paused

I reviewed the current implementation and the README. The main issue is that the previous guard logic was choosing patrol endpoints correctly in many cases, but it was still more complicated than necessary for a procedurally generated map.

For this rewrite, the guard uses a much simpler rule:

- Find the chest.
- Look for **2 or 3 consecutive FLOOR tiles horizontally near it.**
- If none exist, look for **2 or 3 consecutive FLOOR tiles vertically.**
- Prefer candidates away from the starting room/start position.
- The guard moves only along that validated FLOOR segment.
- At each endpoint it snaps exactly to the endpoint and reverses.
- No floating-point movement or `Vector2`.
- Traps remain completely separate and unchanged.

`</>` Python

```
import pygame
import random

TILE = 40
COLS, ROWS = 20, 15
WALL, FLOOR, CHEST, KEY, TRAP = 0, 1, 2, 3, 4
SPEED = 3
GUARD_SPEED = 2

def generate_world():
    grid = [[WALL]*COLS for _ in range(ROWS)]
    rooms = []

    for _ in range(8):
        w = random.randint(3,6)
        h = random.randint(3,5)
        x = random.randint(1, COLS-w-1)
        y = random.randint(1, ROWS-h-1)
        room = pygame.Rect(x, y, w, h)
        overlap = any(room.inflate(2,2).collidect(r) for r in rooms)
```

Task 2 changes only

The guard now uses **validated contiguous FLOOR segments** for its entire patrol. Horizontal segments are searched first near the chest, followed by vertical segments if necessary. Because every tile along

the selected segment is checked as `FLOOR`, the guard cannot cross a wall or trap.

The guard also uses integer pixel movement and snaps exactly to each endpoint, so there is no accumulated floating-point movement or endpoint oscillation.

The Task 1 trap code, key collection, chest completion, player movement, and reset-on-collision behaviour are otherwise unchanged. <>

I need to implement Task 3 from the README in the existing Treasure Hunt game.

Task 3 requirements:

- Add a mini-map to the game.
- The mini-map must show the dungeon layout.
- It must show walls.
- It must show floor.
- It must show the player's current position.
- The player's position on the mini-map must update in real time as the player moves.
- The mini-map should remain visible while playing.
- Keep the main game view and its existing appearance/functionality intact.
- The existing Task 1 floor traps must continue working.
- The existing Task 2 enemy guard must continue working exactly as it currently does.
- The existing key and chest behaviour must remain unchanged.
- Do not implement Task 4 inventory UI yet.
- Do not add external dependencies beyond pygame-ce.
- Do not unnecessarily restructure or rewrite the game.

IMPORTANT:

1. Analyze the current game.py and preserve the existing Tasks 1 and 2 implementation.
2. Add the mini-map as a separate, understandable part of the existing rendering code.
3. The mini-map must accurately represent the current generated dungeon, not a hardcoded map.
4. The mini-map must update every frame so the player's position

moves on it.

5. Clearly distinguish walls and floor on the mini-map.
6. Clearly distinguish the player from the dungeon.
7. Do not regenerate or modify the actual dungeon when drawing the mini-map.
8. Return the COMPLETE updated game.py in ONE code block.
9. The code must be directly runnable with:
python game.py
10. Do not provide a diff or partial snippets.
11. Do not omit any existing functionality.
12. Keep the implementation simple enough for a student to explain during a lab evaluation.
13. After the complete code, briefly explain only what you changed for Task 3.

I reviewed the current Task 2 implementation. I'm keeping the guard implementation exactly as it is and adding the mini-map as a separate `draw_minimap()` method.

The mini-map:

- Uses `self.grid`, so it represents the **actual generated dungeon**.
- Draws `WALL` and all non-wall tiles as floor terrain.
- Draws the player's current tile position every frame.
- Does not modify the grid.
- Does not affect player, trap, guard, key, or chest logic.
- Is drawn as an overlay in the top-right of the existing game area.

`</>` Python

```
import pygame
import random
```

```
TILE = 40
COLS, ROWS = 20, 15
WALL, FLOOR, CHEST, KEY, TRAP = 0, 1, 2, 3, 4
SPEED = 3
GUARD_SPEED = 2
```

```
def generate_world():
    grid = [[WALL]*COLS for _ in range(ROWS)]
```

```
rooms = []

for _ in range(8):
    w = random.randint(3,6)
    h = random.randint(3,5)
    x = random.randint(1, COLS-w-1)
    y = random.randint(1, ROWS-h-1)
    room = pygame.Rect(x, y, w, h)
```

Task 3 changes

Only two things were added:

1. `draw_minimap()`

- Reads the existing `self.grid`.
- Dark tiles represent walls.
- Light tiles represent floor/passable dungeon.
- Blue square represents the player's current tile.
- Nothing in the actual dungeon is modified.

2. One call in `draw()`

`</>` Python

```
self.draw_minimap()
```

Because `draw()` runs every frame, the player's mini-map position updates continuously.

The **Task 1 traps, Task 2 guard, key, chest, player movement, collision handling, and restart behaviour are unchanged.**

I need to implement Task 4 from the README in the existing Treasure Hunt game.

Task 4 requirements:

- Add an inventory UI to the game.
- The inventory should clearly show the important items currently held by the player.
- The key must appear in the inventory once the player collects it.
- The inventory should update when the player's item state changes.
- The inventory should remain visible during gameplay.
- Keep the existing main dungeon view, mini-map, traps, and enemy

guard working.

- Keep the existing key and chest behaviour unchanged.
- Do not remove or replace the mini-map from Task 3.
- Do not add external dependencies beyond pygame-ce.

Gameplay consistency requirements:

- The procedurally generated dungeon must remain playable after traps are placed.
- Trap placement must not completely block the only usable route between important areas such as the starting area, key, and chest.
- Avoid placing a trap in a position that makes an otherwise reachable objective inaccessible.
- Preserve the existing trap behaviour when a player actually steps on one.

IMPORTANT:

1. Analyze the current game.py and preserve Tasks 1, 2, and 3.
2. Implement Task 4 within the existing architecture.
3. Do not unnecessarily redesign the game.
4. Return the COMPLETE updated game.py in ONE code block.
5. The code must be directly runnable with:
`python game.py`
6. Do not provide a diff or partial snippets.
7. Do not omit any existing functionality.
8. Keep the implementation simple enough for a student to explain during a lab evaluation.
9. Do not implement additional features beyond Task 4 and the gameplay-consistency requirements above.
10. After the complete code, briefly explain the changes made.

I'll preserve the current Task 1–3 architecture and only add the inventory plus the trap-placement safeguard. For the gameplay-consistency part, the cleanest approach is to place traps **only when a trap-free path still exists from the start to the key and from the key to the chest**. That means traps remain dangerous, but they cannot accidentally make an objective unreachable.

</> Python

```

import pygame
import random

TILE = 40
COLS, ROWS = 20, 15
WALL, FLOOR, CHEST, KEY, TRAP = 0, 1, 2, 3, 4
SPEED = 3
GUARD_SPEED = 2

def has_path(grid, start, target):
    if start is None or target is None:
        return False

    start_r, start_c = start[1], start[0]
    target_r, target_c = target[1], target[0]

    queue = [(start_r, start_c)]
    visited = {(start_r, start_c)}

    while queue:

```

What changed

Task 4: Added `draw_inventory()` , which displays:

- Inventory: [EMPTY] initially.
- Inventory: [KEY] immediately after the existing key logic sets `self.player.has_key = True` .

Because the inventory is drawn every frame, it automatically reflects the player's current item state.

Gameplay consistency: Trap generation now temporarily places each candidate trap and checks for a **trap-free path from start → key and key → chest**. If either path would disappear, that trap is rejected and the tile is restored to `FLOOR` .

Tasks 1, 2, and 3 otherwise remain intact: traps still reset the player, the guard still patrols/collides normally, and the mini-map still uses the current generated `self.grid` .